Ders 3 Çalışma Özeti

Ders 3 Çalışma Özeti

Genel Konular

- Dersin tarihi 22 Ekim 2020 Perşembe. Ders Zoom üzerinden yapıldı.
  - Asıl anlatım yaklaşık 00:05'te başlar. 01:04 ile 01:18 arasında ara var.
  - Konu kitabın 2. bölümü: Association Rules and Sequential Patterns. Sıralı örüntüler (sequential patterns) kısmına vakit kalmadı.
- Dönem projesi ve değerlendirme hatırlatması
- İlişkilendirme kuralı madenciliğinin (association rule mining) motivasyonu
  - Çapraz satış (cross-sell) ve üst satış (upsell).
  - Market sepeti analizi (market basket analysis), Agrawal, 1993.
- Gösterim: öğe (item), öğe kümesi (itemset), işlem (transaction), işlem veri kümesi.
- Kuralın yapısı $X \to Y$, destek (support) ve güven (confidence).
- İki örnek üzerinde kural çıkarma.
- Web verisine benzetme: belgeler = işlemler, anahtar sözcükler = öğeler.
- Apriori algoritması
  - Aşağı kapanış (downward closure / Apriori) özelliği.
  - Düzey düzey (level-wise) arama, aday üretme (candidate generation): birleştirme (join) ve budama (prune) adımları.
  - Sık öğe kümelerinden kural üretme.
- Veri biçimi dönüşümü (tablo biçiminden işlem biçimine).

Hocanın Özellikle Vurguladığı Kısımlar

- **Dönem projesi:**
  - Notu üç şey belirler: vize, dönem projesi ve final.
  - Dönem içi puanda vize ile projenin katkısı eşittir.
  - Proje 1 ya da 2 kişiyle yapılır.
  - 5. haftada bir sayfalık proje önerisi (project proposal) istenecek. Öneride şunlar yer alacak:
    - grup üyeleri,
    - ne yapılacağı ve neden,
    - veri kümesi ve linki,
    - kitabın hangi bölümüyle ilgili olduğu,
    - nasıl değerlendirileceği,
    - kalan iki aylık süre için haftalık iş planı.
  - Her grup önerisini 5 dakikada sunacak.
- **Bonus puan:** Proje, yerel geliştirme ortamı dışında ölçeklenebilir bir bulut ortamında (Amazon AWS, Azure, Google Cloud) sanallaştırma teknolojileriyle (Docker, Docker Swarm vb.) çalıştırılıp gösterilirse bonus verilecek.
- **Güven ve destek farkı:** Bir kuralın support değeri, kuralın iki tarafı yer değiştirince değişmez. Confidence değeri ise değişir.
- **Pratikte:** Kurallar genellikle yüksek confidence ama düşük support değerine sahiptir.
- **Kural üretmek için veriye geri dönmeye gerek yok:** Apriori çalışırken tüm alt kümelerin support değerleri zaten hesaplanmıştır. Bu değerler bir tabloda (hash table) tutulur, confidence buradan hesaplanır. Hoca "bu kısım önemli" diye vurgular.
- **Asıl maliyet:** Algoritmanın her geçişinde veritabanına disk erişimi (I/O) yapılır ve tüm işlemler sırayla okunur. Zamanı alan kısım budur.

- k**'nin sınırı:** Pratikte en büyük öğe kümesi boyutu sınırlanır (hocaya göre genellikle 10). Zaten 2–4 öğelik birliktelikleri bulmak çoğu iş için yeterlidir.

Kısa Tekrar Notları

- $I = \{i_1, \ldots, i_m\}$: mağazadaki tüm öğeler.
- $T = \{t_1, \ldots, t_n\}$: işlemler. Her işlem bir öğe kümesidir ($t_i \subseteq I$) ve bir kimliği (TID) vardır.
- Kural: $X \to Y$, burada $X, Y \subset I$ ve $X \cap Y = \emptyset$. k öğeli küme k-itemset diye adlandırılır.
- Destek ve güven:
  - $\text{support}(X \to Y) = \dfrac{(X \cup Y)\text{'yi içeren işlem sayısı}}{n}$
  - $\text{confidence}(X \to Y) = \dfrac{(X \cup Y)\text{'yi içeren işlem sayısı}}{X\text{'i içeren işlem sayısı}} = \dfrac{\text{support}(X \cup Y)}{\text{support}(X)}$
  - Buradaki $X \cup Y$, "hem X'in hem Y'nin tüm öğelerini içeren işlem" demektir. "X'i ya da Y'yi içeren" anlamına gelmez.
- Amaç: minsup ve minconf eşiklerini geçen tüm kuralları bulmak. Kuralın sağ tarafında sınıf etiketi yoktur; iki taraf da öğedir.
- Aşağı kapanış özelliği: Bir öğe kümesi sıksa, tüm alt kümeleri de sıktır. Bir alt kümesi sık değilse, kümenin kendisi de sık olamaz.
- Apriori adımları:
  1. Tek öğeleri say, F_1'i bul.
  2. $k = 2, 3, \ldots$ için F_{k-1}'den adaylar C_k'yı üret (join), alt kümesi F_{k-1}'de olmayan adayları at (prune).
  3. Veriyi tekrar tara, adayları say, F_k'yı bul.
- k öğelik kümeleri bulmak için veri üzerinde k geçiş yapılır.

Detaylı Açıklamalar

Motivasyon: çapraz satış ve üst satış

- **Cross-sell:** Çok satılan bir ürün, az satılan bir ürünle aynı pakette indirimli satılır. Müşteri popüler ürünü almak ister. Paketteki ikinci ürünü pek umursamaz.
- **Upsell:** Özellikle hizmet sektöründe kullanılır. Müşteri, kullandığı üyelik seviyesinden daha üst bir seviyeye uygun fiyatla geçirilmeye çalışılır.
- Pazarlamacılar bu stratejiler için bilişimcilerden birlikte sık satılan ürünleri bulmalarını bekler.
- Agrawal 1993'te problemi "market sepeti analizi" diye adlandırdı. Çok sayıda müşteri bazı ürünleri birlikte alıyorsa bu ürünler ilişkilidir. Bunlar mağazada yan yana konabilir ya da çapraz satışa konu olabilir.
- Örnek kural: Bread $\to$ Milk [sup = %5, conf = %100].
  - %5 destek: tüm işlemlerin %5'inde ekmek ve süt birlikte var.
  - Confidence, kurala ne kadar güvenebileceğimizi söyler: ekmek içeren işlemlerin ne kadarında süt de var?

Problem önce fiziksel mağazalarda ortaya çıktı. Sonra e-ticaret sitelerine taşındı. Ardından web madenciliğine uyarlandı. Bu uyarlamada her belge bir işlem, belgedeki her anahtar sözcük bir öğe sayılır. Böylece bir belge kümesinde en sık birlikte geçen anahtar sözcükler bulunur.

İşlem verisinde genellikle miktar ve fiyat gibi ek nitelikler de vardır. Yeni algoritmalar bunları da kullanır. Derste basitleştirilmiş bakış kullanılır: işlem kimliği ve öğe kimlikleri.

Örnek 1 (minsup = %50, minconf = %50)

TID	Alınan öğeler
10	A, B, D
20	A, C, D
30	A, D, E
40	B, E, F

TID	Alınan öğeler
50	B, C, D, E, F

- Sık örüntüler: $\{A:3,\ B:3,\ D:4,\ E:3,\ AD:3\}$. C ve F yalnız 2 işlemde geçer (%40), elenir.
- Kural yazmak için en az iki öğeli bir küme gerekir. Burada yalnız $\{A, D\}$ var.
  - $A \to D$: support $3/5 = \%60$, confidence $3/3 = \%100$.
  - $D \to A$: support $3/5 = \%60$, confidence $3/4 = \%75$.
- İki kuralın support değeri aynıdır, confidence değeri farklıdır. İkisi de eşiği geçer.

Python ile sayımlar kontrol edildi; slayttaki değerlerle aynı çıktı.

Bir öğrenci support'un neden %80 değil de %60 çıktığını sordu. Formüldeki birleşim ($X \cup Y$) ifadesini "A ya da D'yi içeren işlemler" diye anlamıştı. Hoca açıkladı: birleşim, kuralın iki tarafındaki öğelerin hepsini birlikte içeren işlemleri ifade eder. $\{A, D\}$'yi içeren işlem sayısı 3, toplam işlem 5, yani $3/5$.

Örnek 2 (minsup = %30, minconf = %80)

İşlemler: - t1: Beef, Chicken, Milk - t2: Beef, Cheese - t3: Cheese, Boots - t4: Beef, Chicken, Cheese - t5: Beef, Chicken, Clothes, Cheese, Milk - t6: Chicken, Clothes, Milk - t7: Chicken, Milk, Clothes

En büyük sık öğe kümesi $\{$Chicken, Clothes, Milk$\}$, support $3/7$ (t5, t6, t7). Bu kümeden çıkan tüm kuralların support değeri $3/7$'dir. Confidence değerleri:

Kural	Confidence	minconf %80
Clothes → Milk, Chicken	$3/3 = \%100$	geçer
Clothes, Chicken → Milk	$3/3 = \%100$	geçer
Clothes, Milk → Chicken	$3/3 = \%100$	geçer
Milk → Chicken, Clothes	$3/4 = \%75$	geçmez
Chicken, Milk → Clothes	$3/4 = \%75$	geçmez
Chicken → Clothes, Milk	$3/5 = \%60$	geçmez

Bir öğrenci "Chicken, Milk → Clothes" kuralını sordu. Hoca hesapladı: Chicken ve Milk birlikte 4 işlemde var, üçünde Clothes da var. Güven %75 olduğu için kural alınmaz. Yorum: Clothes alan herkes Milk ve Chicken da almış, bu kurala %100 güvenilir. Milk ve Chicken alanların ise ancak %75'i Clothes almış.

Apriori algoritması

Girdi: işlem veri kümesi T, minsup ve minconf. Bu girdiler için kural kümesi tek türlü belirlenir. Algoritma iki adımlıdır: 1. Support değeri minsup'ı geçen tüm sık öğe kümelerini bul. 2. Sık öğe kümelerinden minconf'u geçen kuralları üret.

Sık olma şartları: - Öğe kümesinin support değeri minsup'ın üzerinde olmalı. - Tüm alt kümeleri de sık olmalı (aşağı kapanış özelliği).

Algoritma yinelemeli (iterative) ve aşağıdan yukarıya, düzey düzey çalışır. F_k gerçek sık k-öğe kümelerini, C_k aday kümeleri gösterir.

Örnek (minsup = 0,5, yani en az 2 işlem):

TID	Öğeler
T100	1, 3, 4
T200	2, 3, 5
T300	1, 2, 3, 5
T400	2, 5

TID	Öğeler

1. 1. tarama: C_1: {1}:2, {2}:3, {3}:3, {4}:1, {5}:3. Buradan F_1: {1}, {2}, {3}, {5}.
2. C_2: {1,2}, {1,3}, {1,5}, {2,3}, {2,5}, {3,5}.
3. 2. tarama: {1,2}:1, {1,3}:2, {1,5}:1, {2,3}:2, {2,5}:3, {3,5}:2. Buradan F_2: {1,3}, {2,3}, {2,5}, {3,5}.
4. C_3: {2,3,5}.
5. 3. tarama: {2,3,5}:2. Buradan F_3: {2,3,5}.

Python ile çalıştırıldı; her adımın sayımları slayttakiyle aynı çıktı.

Budama (prune): Hoca F_2'den üretilebilecek $\{1,2,3\}$ ve $\{1,3,5\}$ adaylarını ele alır. $\{1,2\}$ ve $\{1,5\}$ sık olmadığı için bu adaylar veriye bakmadan atılır. Böylece 3. taramada daha az aday sayılır.

Hocanın ikinci budama örneği: $F_3 = \{\{1,2,3\},\{1,2,4\},\{1,3,4\},\{1,3,5\},\{2,3,4\}\}$. Birleştirmeden sonra slaytta $C_4 = \{\{1,2,3,4\},\{1,2,3,5\},\{1,3,4,5\}\}$ adayları var. - $\{1,2,3,4\}$: tüm 3'lü alt kümeleri F_3'te var, kalır. - $\{1,2,3,5\}$: $\{2,3,5\}$ F_3'te yok, atılır. - $\{1,3,4,5\}$: $\{1,4,5\}$ (ve $\{3,4,5\}$) F_3'te yok, atılır. - Budamadan sonra $C_4 = \{\{1,2,3,4\}\}$. Fonksiyon tek aday döndürür.

```
Algorithm Apriori(T)
  C1 <- init-pass(T)                      // tek öğeleri say
  F1 <- {f | f in C1, f.count/n >= minsup}
  for (k = 2; F(k-1) != {}; k++) do
      Ck <- candidate-gen(F(k-1))         // join + prune
      for each transaction t in T do      // k. tarama
          for each candidate c in Ck do
              if c is contained in t then c.count++
      Fk <- {c in Ck | c.count/n >= minsup}
  return F <- union of all Fk
```

Sık öğe kümesinden kural üretme

X sık bir öğe kümesiyse, X'in her boş olmayan öz alt kümesi A için $A \to (X-A)$ kuralı yazılır. - Support değeri hep support(X)'tir. - Confidence değeri support(X)/support(A)'dır.

Bu hesap için toplam işlem sayısına ya da veriye tekrar dönmeye gerek yoktur.

Örnek: $\{2,3,4\}$ sık, support %50. Öz alt kümelerin support değerleri: - $\{2,3\}$: %50 - $\{2,4\}$: %50 - $\{3,4\}$: %75 - $\{2\}$, $\{3\}$, $\{4\}$: %75

Kural	Confidence
2,3 → 4	$50/50 = \%100$
2,4 → 3	$50/50 = \%100$
3,4 → 2	$50/75 \approx \%67$
2 → 3,4	$\approx \%67$
3 → 2,4	$\approx \%67$
4 → 2,3	$\approx \%67$

- minconf %50 ise altı kural da alınır.
- minconf %80 ise yalnız ilk iki kural alınır.

Değerler Python ile hesaplanıp doğrulandı.

Pratik notlar

- Gerçek işlem verisinde milyonlarca (Amazon gibi sitelerde milyarlarca) işlem ve binlerce, milyonlarca ürün vardır. Algoritmalar hem ölçeklenebilir hem doğru olmalıdır.
- Hoca birlikte satılan ürünlerin bilinmesinin mağazada nasıl kullanılabileceğini anlatır:

  - Ürünler yan yana konur ki kolay alınsın.
  - Ya da ürünler uzak noktalara konur ki müşteri aradaki ürünleri de görsün.
  - Ya da popüler üçlünün yanına az satılan dördüncü bir ürün eklenip çapraz satış yapılır.
- **Veri biçimi:** Apriori her satırı bir işlem, her değeri bir öğe olan biçim ister. Veri tablo biçimindeyse (nitelik-değer çiftleri) önce `(Attribute1, a)`, `(Attribute2, b)` gibi öğelere dönüştürülür.
- Sıralı örüntü madenciliği bir sonraki derse kaldı.

Ders sonrası notlar

- **Slayttaki confidence formülü:** Ara sırasında bir öğrenci, slayttaki koşullu olasılık formülünün payının yanlış olduğunu belirtti. Hoca kabul etti ve slaytı düzelteceğini söyledi. Doğrusu: $\text{conf}(X \to Y) = P(Y \mid X) = \dfrac{\text{count}(X \cup Y)}{\text{count}(X)}$. Pay, hem X'in hem Y'nin öğelerini içeren işlemleri sayar.
- **Formüldeki küçük sürçme:** 01:54 civarında hoca iki kez "denominator" (payda) der. Doğrusu: birleşimin support değeri, yani $\text{support}(X \cup Y)$, paydır. Paydada sol tarafın support değeri bulunur.
- **Birleştirme (join) adımı:** Hoca ilk örnekte $\{1, 2, 3\}$ ve $\{1, 3, 5\}$'in aday olarak üretilip sonra budandığını anlatır. Kitaptaki `candidate-gen` ise yalnız ilk $k - 2$ öğesi aynı olan iki kümeyi birleştirir. Bu kuralla $\{1, 3\}$'ün birleşeceği bir eş yoktur. Yalnız $\{2, 3\}$ ile $\{2, 5\}$ birleşip $\{2, 3, 5\}$ üretilir. Aynı nedenle ikinci örnekteki $\{1, 2, 3, 5\}$ de kitaptaki birleştirmede hiç üretilmez. Sonuç aynıdır, budama mantığı da doğrudur; fark yalnız ara adaylardadır.
- **"İteratif mi, özyinelemeli mi":** Hoca bir an özyinelemeli (recursive) algoritmadan söz eder. Apriori özyinelemeli değil, döngüyle çalışan yinelemeli bir algoritmadır.
- **minsup = %50 yorumu:** 4 işlemde %50, "en az 2 işlemde geçmeli" demektir. Altyazıdaki "at least one item" ifadesi sürçmedir.
- **k'nin üst sınırı:** "Genellikle 10" değeri hocanın pratik önerisidir. Algoritmanın kendisinde böyle bir sınır yoktur. Algoritma, aday kalmayınca durur.
- **Altyazı hataları:**
  - 01:58–02:00 arasında tekrar eden "the three of the three" satırları ve bozuk İngilizce cümleler, hocanın Türkçe anlattığı kısmın hatalı çevirisidir.
  - "Agrava" Agrawal, "AppRory" Apriori, "download closure" downward closure, "precooked K item item sets" frequent k-itemsets demektir.

Not: İsterseniz bu dersin altyazı (.srt) dosyasını NotebookLM gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz.